

24/8/26-

main()

```
{
    int A, B, C;
    A = 15;
    B = 7;
    C = A + B;
    printf("Add");
    printf("%d", C);
}
```

① int A → Declaration → A is a variable containing int type
② A = 15 → Definition of value is value of
* ③ int i = 7; → initialization

25/8/26-

main()

```
{
    float n1, n2;
    float Sum;

```

n1	n2	Sum
7.1	9.9	

n1 = 7.1;

n2 = 9.9;

```
printf("Add");
printf("%f", Sum); }
```

printf("Sum of %f", n1, n2, Sum) α
printf("Sum of %f and %f is %f", n1, n2, Sum)

n1	n2	Sum
7.3	6.2	13.5

Sum of 7.3 and 6.2 is 13.5

Size of int = 2 Byte.
float = 4 Byte.
char = 1 Byte

31/8/2021

```
main ()
{
    float div;
    int A, B, Sum, sub, mul; div;
    A = 1;
    B = 2;
    Sum = A + B;
    sub = B - A;
    mul = A * B;
    div = B / A;
    printf("Sum of %d and %d is %d", A, B, Sum);
    printf("Sub of %d and %d is %d", B, A, sub);
    printf("mul. of %d and %d is %d", A, B, mul);
    printf("div. of %d and %d is %f", B, A, div);
}
```

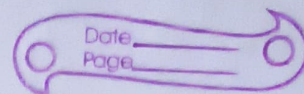
Table of 4:-

```
main ()
{
    int A, n1, n2, n3, n4, n5, n6, n7, n8, n9, n10; x1, x2,
    x3, x4, x5, x6, x7, x8, x9, x10;
    A = 4;
    n1 = 1;
    n2 = 2;
    n3 = 3;
    n4 = 4;
    n5 = 5;
    n6 = 6;
    n7 = 7;
    n8 = 8;
    n9 = 9;
    n10 = 10;
}
```

For Optimization of Code- ① Storage

② Line of Code (LOC)

③ Execution (Output).



$x1 = A \times n1;$

$x2 = A \times n2;$

$x3 = A \times n3;$

$x4 = A \times n4;$

$x5 = A \times n5;$

$x6 = A \times n6;$

$x7 = A \times n7;$

$x8 = A \times n8;$

$x9 = A \times n9;$

$x10 = A \times n10;$

~~$\text{printf}(x1, x2, x3, x4, x5, x6, x7, x8, x9, x10);$~~

$\text{printf}(\text{"%.d, %.d, %.d, %.d, %.d, %.d, %.d, %.d, %.d, %.d", } x1, x2, x3, x4,$
 $x5, x6, x7, x8, x9, x10);$

}



main()

{ int A, n;

A = 4;

n = A * 1;

~~$\text{printf}(\text{"%.d", } n);$~~

n = A * 2;

~~$\text{printf}(\text{"%.d", } n);$~~

n = A * 3;

~~$\text{printf}(\text{"%.d", } n);$~~

n = A * 4;

~~$\text{printf}(\text{"%.d", } n);$~~

n = A * 5;

~~$\text{printf}(\text{"%.d", } n);$~~

```

n = A * 6;
printf("%d", n);
n = A * 7;
printf("%d", n);
n = A * 8;
printf("%d", n);
n = A * 9;
printf("%d", n);
n = A * 10;
printf("%d", n); }

```

1/9/26)

```

scanf("%d", &n1)
           ↓
        address of.

```

main()

```

{ int n1, n2, Sum;

```

```

    scanf("First Digit", &n1)
    scanf("Second Digit", &n2)

```

```

    printf("n1");
    scanf("%d", &n1);
    printf("n2");
    scanf("%d", &n2);
    Sum = n1 + n2;
    printf("Sum is %d", Sum);
}

```

2/9/26)

	$i = 2;$		$if (<Condition>)$ replacable.
	$if (i == 1)$		{
T.B	{ $printf("one");$ }		True Block
	else		{ False Block }
F.B	{ $printf("not one");$ }		

7/9/26)

lib. files $\rightarrow$ #include $\leftarrow$ stdio.h $\leftarrow$ $\begin{matrix} \text{std. input} \\ \text{output} \end{matrix}$
Pre Processor

$void main() \leftarrow func$

{ $int n_1, n_2;$ $\leftarrow$ Variable declaration.

For	n_1	n_2	
	7	3	✓ Works
	3	7	✓ Works.
	5	5	X Don't Work.

$func \leftarrow$ $printf("Enter n_1 and n_2");$
 $\leftarrow$ $scanf("%d %d", &n_1, &n_2);$
 formula specifics $\leftarrow$ $\leftarrow$ add of
 $if (n_1 > n_2)$
 { $printf("n_1 is greater");$
 }
 else
 { $printf("n_2 is greater");$
 }
 }

I) if ($n_1 == n_2$)
{ printf("n1 and n2 are equal")
}

For n_1 n_2
7 3 ✓
3 7 ✓
5 5 ✓ } work.

if ($n_1 > n_2$)
{ printf("n1 is greater than n2")
}

if ($n_2 > n_1$)
{ printf("n2 is greater than n1")
}

II) if ($n_1 == n_2$)
{
 ① eq.
}

$== \rightarrow$ AND ; $|| \rightarrow$ OR
 $!= \rightarrow$ Not equal.

else
{
 if ($n_1 > n_2$)
 { n_1
 }
 else { n_2
 }
}

Divisible by no. 1.

~~if ($x \geq 0$)~~
~~{ printf("Divisible")~~

A no. divisible by other no:-

```
{ a, x, i;
  printf("Enter the no.");
  scanf("%d", &a);
  printf("Enter the divisor");
  scanf("%d", &i);
  x = a%i;

  if (x == 0)
  { printf("Divisible");
  }
  else { printf("Not Divisible");
  }
  return 0;
}
```

8/9/26 - Greatest of 3 no. -

include <stdio.h>

int main()

{ int n1, n2, n3;

printf("Give 1st No. - ");

scanf("%d", &n1);

printf("Give 2nd No. - ");

scanf("%d", &n2);

printf("Give 3rd No. - ");

scanf("%d", &n3);

if (n1 > n2)

{ if (n1 > n3)

{ printf("1st is greater");
}

}

if (n2 > n1)

{ if (n2 > n3)

{ printf("2nd is greater");
}

}

if (n3 > n1)

{ if (n3 > n2)

{ printf("3rd is greater");
}

}

return 0;

}

n1 n2 n3

2 5 8

8 2 5

5 8 2

if (n1 > 2 && n1 > 5)

{ printf("1st");
}

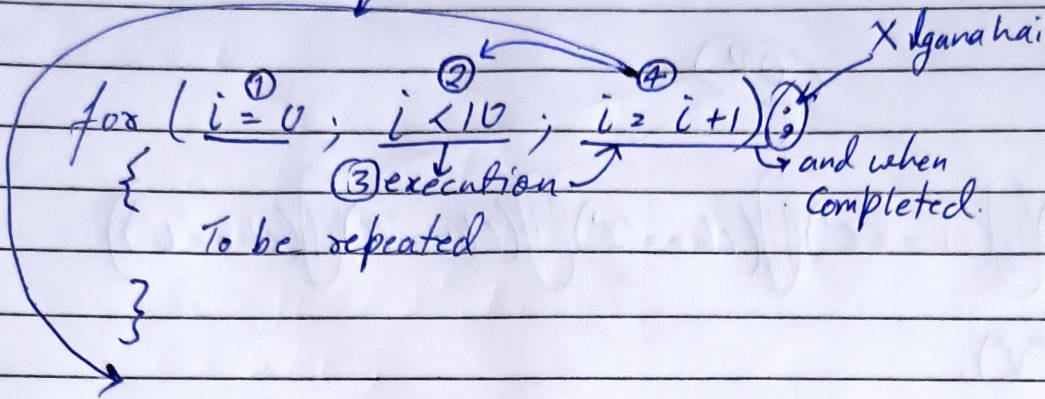
if (n2 > 2 && n2 > 5)

{ printf("2nd");
}

if (n3 > 2 && n3 > 5)

{ printf("3rd");
}

14/9/26 - Loops - for while do while.



⇒ Escape Sequence → like \n.

15/9/26 -

```
for ( i = 1 ; i < 16 ; i = i + 1 )
{
    printf("%d", i);
}
```

Switch-Case

```
Switch (x)
{
    case 0...50
    {
        break;
    }
    case 1
    {
        break;
    }
}
```

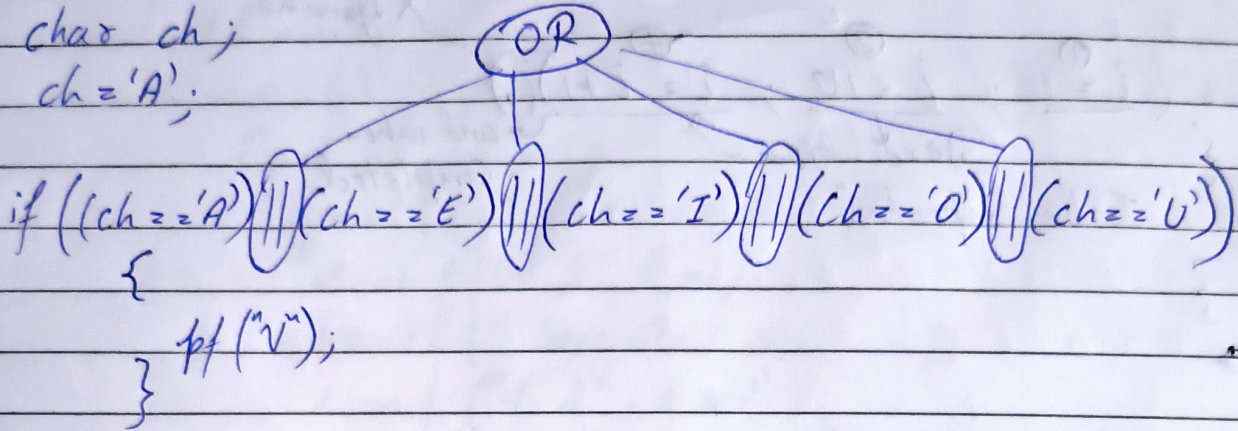
Annotations:

- Range: points to the range of values for the switch variable `x`.
- `x`: points to the switch variable.
- `0...50`: points to the first case range.
- `break;`: points to the break statement in the first case.
- `break;`: points to the break statement in the second case.

Agar nhi lgaye, to Case Tunc
uske age wale save
Tunc.

16/2/261-

char ch;
ch = 'A';



How the code runs:-

$\begin{matrix} s & e \\ \boxed{2} & \boxed{15} \end{matrix}$

```

for (i = s; i <= e; i++)
{
    i = i + 1 → i--
    (i = i - 1)
    if (i % 2 == 0)
    {
        pf()
        cnt = cnt + 1 ← for counting.
    }
}
  
```

4 3 2 1
 ↓
 4 × 1000
 3 × 100
 2 × 10
 1 × 1

1 2 3 4
 ↓
 1 × 1000
 2 × 100
 3 × 10
 4 × 1

$x = f(2, 3)$

$x = f(2, 3)$

$x = y^v$

$x = g+2$; Exp

28/9/26 1-

$x = 2$; True/Non-Zero.
 if ($x = 0$)
 {
 }
 instead of "0" every.
 no is Non-Zero.

False/Zero.
 if ($x = 0$)
 {
 }

if ($x > 50$)
 {
 }
 if ($!(x < 50)$)
 {
 }
 if (x)

29/9/26 1-

void addf();
 main()
 {
 }
 addf();
 }

void addf()
 {
 int n1, n2, R;
 printf("N1");
 scanf("%d", &n1);
 R = n1 + n2;
 printf("%d", R);
 }

Syntax of Defⁿ:-

<dt> <fname>()
 {
 fn body.
 }